

Assignment Report

I tested three ASR systems on one question; can they correctly identify a Bangalore locality name from a naturally spoken, noisy phone quality audio clip.

Recordings

20 audio clips recorded on an iPhone mic. Each clip is a natural Hindi, English or Hinglish sentence with a Bangalore locality name in it; the kind of thing a candidate would actually say on a call, not a clean pronunciation of the locality name.

I varied the recording conditions to simulate real phone call environments:

Condition	Localities
Clean room	Koramangala, Indiranagar, Whitefield, Electronic City
Guitar in background	Marathahalli, Jayanagar
People talking nearby	Rajajinagar, Hebbal
Music in background	Banashankari
Traffic noise	Yelahanka, HSR Layout, BTM Layout, Bellandur
Fan sound	Sarjapur
Crowded place	Hesaraghatta, Rajarajeshwarinagar, Silk Board
Car horn	KR Puram
Kid crying	Thalaghattapura
Announcement in background	Thanisandra

Files were recorded as m4a on the iPhone and converted to wav using ffmpeg before passing to the models.

Models

I chose three models.

Deepgram Nova-2 is the baseline. Standard production ASR API, widely used in call center stacks. Fast and easy to integrate.

Sarvam Saaras V3 is built specifically for Indian languages and telephony audio. The only model here that was actually designed for this problem. Trained on 22 Indian languages including Hindi.

Whisper medium is OpenAI's open source multilingual model. Runs locally.

Additional Dataset

I also tested all three models on 20 samples from the FLEURS Hindi dataset; clean, studio quality Hindi audio with no background noise. If models score much better on FLEURS than on my recordings, noise is the bottleneck, not the models.

FLEURS is freely available on HuggingFace.

Metrics

7 metrics total. WER is included for comparability with published benchmarks but is not the main metric.

WER is how many words were wrong in the full sentence. The problem with WER is it treats "bhai" and "Koramangala" as equally important errors; which is wrong for this use case.

MER is match error rate, similar to WER but fairer. Does not penalise short sentences harshly.

WIL is word information lost; how much of the original sentence was lost in the transcript. If the locality name is lost, WIL goes high even if WER looks okay.

NE-WER is WER computed only on the locality name, not the full sentence. Uses a sliding window to find the best matching region in the transcript.

Jaro-Winkler is a fuzzy similarity score on just the locality name. Catches near misses like "herbell" vs "hebbal" that WER would just mark as wrong.

ERUN is Entity Recall Under Noise. Did the model capture the locality name well enough to be usable? A clip counts as a hit if the Jaro-Winkler score is above 0.80.

Hallucination Rate is the percentage of clips where the model output has less than 10% word overlap with the reference. Based on the SHALLOW benchmark framework (2025).

One important caveat; all three models return transcripts in Devanagari script, but reference sentence is in English letters. I converted model output to Roman using the indic-transliteration library before computing metrics. This conversion is imperfect for code-mixed Hinglish sentences, so the metrics are directional comparisons, not exact scores. The ranking between models is reliable; the absolute numbers are not.

Results

My Recordings

Metric	Deepgram	Sarvam	Whisper
WER	0.55	0.99	0.93
MER	0.49	0.89	0.90
WIL	0.66	0.97	0.96
ERUN	80%	70%	55%
Hallucination	0%	50%	55%
Latency	0.97s	2.2s	>90s

Deepgram wins on every metric; 80% ERUN, 0% hallucination, and the only model under 1 second latency.

Sarvam and Whisper show high hallucination rates of 50 to 55%. This is partly inflated by the transliteration step; Hindi script does not convert cleanly to Roman for code-mixed sentences, so word overlap drops artificially. Their true hallucination rate is likely lower.

Whisper's >90 second latency was on CPU. On GPU it would be faster but still not viable for real-time calls.

FLEURS vs My Recordings

Model	Noisy WER	Clean FLEURS WER
Deepgram	0.55	0.15
Sarvam	0.99	0.13
Whisper	0.93	0.43

All three models perform much better on clean audio. Deepgram drops from 0.55 to 0.15; Sarvam drops from 0.99 to 0.13. This confirms that noise is the bottleneck, not the models. On clean audio Sarvam actually beats Deepgram on WER. Under noise, Deepgram holds up much better.

Whisper's FLEURS WER of 0.43 is notably higher than the other two even on clean audio; it genuinely struggles with Hindi.

Failure Analysis

HSR Layout failed across all 3 models. Heavy sustained traffic noise caused complete collapse; Deepgram output repeated garbage text, Sarvam and Whisper both produced unrelated outputs. This is the only clip where all three simultaneously failed. Loud sustained noise at a consistent frequency seems to break model confidence entirely rather than just degrading it.

Hebbal failed on Deepgram and Whisper; Deepgram output "herbell". Sarvam handled it correctly, likely because its India specific training data included this name.

Thalaghattapura failed on Deepgram; output was "kala ghatta pura". Long unfamiliar South Indian place names are a consistent weak point for generic models not trained on South Indian geography.

Marathahalli on Deepgram output "mara thandi"; a hallucination, not a transcription error. Phonetically distant and semantically wrong.

Approximate Cost at Scale

If we were to handle 500 to 1000 calls a day, that's roughly 17 to 33 hours of audio. Sarvam at ₹30/hour comes to around ₹500 to ₹1000 a day or ₹15,000 to ₹30,000 a month. Deepgram at \$0.0077/minute works out to about ₹800 to ₹1600 a day, roughly 25% more. Whisper is free but GPU infrastructure brings the cost to a similar ballpark anyway.

Recommendation

Deepgram. Best ERUN at 80%, zero hallucinations, only model under 1 second. For real-time calls that's what matters. Sarvam is cheaper and beats Deepgram on clean audio; i'd keep it as a fallback and revisit as it improves. I would suggest fine-tuning on Bangalore locality names. HSR Layout, Hesaraghatta, Thalaghattapura failed because the models simply don't know these names. 500 to 1000 clips would push ERUN from 80% to 90%+.

Limitations

Whisper was tested in batch mode only; streaming latency was not measured. All recordings are from a single speaker so performance across different accents and genders was not evaluated. Utterance lengths were not explicitly analyzed but all clips were between 3 to 6 seconds, which is typical for a candidate answering a location question on a call.